PHASE 5 — Onboarding Implementation Report

Project: WeGlow · Branch: `phase-5-onboarding` · No commits, pushes, merges, resets, or branch switches were made. Working tree contains only the changes documented below.

A. Phase 5 Implementation Result

Phase 5 is implemented against the current project.

- New account / first-time authenticated user runs the **existing** onboarding question screens (no UI redesign). Every answer is held in `OnboardingViewModel.uiState` across all screens.
- On the final question, the answers are assembled into one `UserProfile` keyed to `authRepository.currentUserId()` and persisted with a single Supabase `upsert` to `public.profiles`.
- Onboarding completion is now a **real** `onboarding_completed boolean NOT NULL DEFAULT false` column, written `true` **only** on the success branch of the save. On failure nothing is marked and no forward navigation occurs — the existing error / disabled-button behavior on `SkinSensitivityScreen` is retained.
- Returning authenticated users are routed by the actual `onboarding_completed` value (not by "a row exists"), both at app startup and immediately after authentication.
- All four onboarding entry paths (manual email signup, email sign-in of an incomplete user, Google auth for a new / incomplete user, restored session with incomplete onboarding) reset `OnboardingViewModel` and resolve the user's identity before the first question, so stale answers from a previous attempt or another user cannot survive, and the manual-signup full name is never clobbered.
- The manual-signup full name is preserved and persisted; for Google users the display name is taken from authenticated user metadata (never fabricated) and persisted when present.
- Home and Profile show the real persisted name instead of the hardcoded "Team 03", loaded through a `ProfileRepository` read; neutral fallbacks are used when no name is available. Layout / styling / navigation of both screens is unchanged. No profile-photo work (that is Phase 6).
- Supabase stays entirely under `data/`. No `io.github.jan.supabase` import was added to `feature/`, `ui/`, or `domain/`. `architectureCheck` still passes. `AppContainer` / `ViewModelFactory` architecture is preserved (one new ViewModel wired identically to the existing five).
- No Supabase credentials are hardcoded. No fake persistence or fake success states.
- Scan / acne model / ONNX / `ScanViewModel` / `ScanScreen` / `ScanResultScreen` / backend / acne tests were not modified.

Verification: `architectureCheck`, `compileDebugKotlin`, `compileDebugUnitTestKotlin`, `lintDebug` (0 errors), and `assembleDebug` (APK produced) all **succeed**. `testDebugUnitTest` is **33 pass / 1 fail**; the one failure is a **pre-existing, previously-masked teammate scan test** (`YoloPostProcessorTest`), unrelated to Phase 5 — see sections K and M. A full PASS is **not** claimed, and scan code was not modified to force one.

B. Every File Created

File	Purpose
`supabase/migrations/20260910000000_add_onboarding_completed.sql`	New, additive migration: adds `onboarding_completed` to `public.profiles` and runs a conservative, evidence-based backfill. Leaves the existing `20260909000000_create_profiles.sql` untouched.
`app/src/main/java/com/example/weglow/feature/profile/ProfileViewModel.kt`	Loads the signed-in user's persisted profile so Home / Profile can display the real name. Depends only on `AuthRepository` + `ProfileRepository` domain interfaces — no Supabase import.
`app/src/test/java/com/example/weglow/data/repository/SupabaseProfileRepositoryTest.kt`	Pure unit tests for the completion-decision rule (flag, not row existence) and the `ProfileRow` ↔ `UserProfile` mappers.

`docs/phase-5/` also appears as untracked — that directory holds the earlier `PHASE_5_AUDIT.md` document, which predates this implementation and is not code.

C. Every File Modified

File	Change
`domain/model/UserProfile.kt`	Added `val onboardingCompleted: Boolean = false` (backend-agnostic domain model).

`domain/repository/ProfileRepository.kt`	Added `suspend fun getProfile(userId: String): Result<UserProfile?>`; documented that `hasCompletedOnboarding` reads the flag, not row existence.
`domain/repository/AuthRepository.kt`	Added `fun currentUserDisplayName(): String?` — best-effort name from the identity provider's metadata; never a UI-supplied or fabricated value.
`data/model/ProfileRow.kt`	Added `val onboarding_completed: Boolean = false`; `toProfileRow()` maps `onboardingCompleted`; new `fun ProfileRow.toUserProfile()` reverse mapper.
`data/repository/SupabaseProfileRepository.kt`	Implemented `getProfile` (`select … decodeList<ProfileRow>().firstOrNull()?.toUserProfile()`). `hasCompletedOnboarding` now returns `List<ProfileRow>.isOnboardingCompleted()` = `firstOrNull()?.onboarding_completed == true` (pure `internal` helper, unit-testable without a client).
`data/repository/SupabaseAuthRepository.kt`	Implemented `currentUserDisplayName()` reading `client.auth.currentUserOrNull()?.userMetadata` keys `full_name` → `name` → `user_name` → `preferred_username` (first non-blank, trimmed). Added imports `kotlinx.serialization.json.JsonPrimitive`, `kotlinx.serialization.json.contentOrNull`.
`feature/onboarding/OnboardingViewModel.kt`	`start(fullName: String? = null)`: always resets `OnboardingUiState`; resolves name from the passed value, else `authRepository.currentUserDisplayName()`, else `""`. The `UserProfile` passed to `saveProfile` now sets `onboardingCompleted = true`; `saveCompleted` / navigation still only occur inside `onSuccess`; failure path unchanged.
`navigation/WeGlowApp.kt`	Added `profileViewModel` (created via `viewModelFactory` from `container.authRepository` + `container.profileRepository`) and `profileState`. Calls `onboardingViewModel.start()` on the restored-session `ONBOARDING` branch and the post-auth `SignedIn` →not-HOME branch (manual-signup path already called `start(event.fullName)` and is unchanged). Home and Profile destinations call `profileViewModel.refresh()` on entry and pass `displayName = profileState.displayName`. All scan navigation left exactly as-is.
`ui/screens/HomeScreen.kt`	Added `displayName: String? = null` param. `"Hello, Team 03"` → `displayName?.let { "Hello, $it" } ?: "Hello there"`. Only the string changed.
`ui/screens/ProfileScreen.kt`	Added `displayName: String? = null` param. `"Team 03"` → `displayName ?: "Your Profile"`. Only the string changed.
`test/…/ArchitectureFoundationTest.kt`	Kept all existing assertions; added `onboardingCompleted = true` to the fixture and `assertEquals(true, row.onboarding_completed)`.
`test/…/feature/auth/AuthViewModelTest.kt`	`FakeAuthRepository` implements `currentUserDisplayName()`; `FakeProfileRepository` implements `getProfile()`, gains an optional `storedProfile`, and derives completion from `storedProfile?.onboardingCompleted ?: completed`. Added test `restoredSession_withProfileRowButIncompleteOnboarding_routesToOnboarding`. All 8 original tests retained (9 total).
`test/…/feature/onboarding/OnboardingViewModelTest.kt`	Fakes rebuilt to the current Phase 4 `AuthRepository` contract (`authenticationState`, `signInWithGoogle`) plus `currentUserDisplayName`; `RecordingProfileRepository` implements `getProfile`, supports an injectable failure `Result`, and records both the attempted and the persisted profile. Added Phase 5 test cases (see section J). This fixes audit blocker B4 — the unit-test source set previously did not compile.
`docs/phase-0/DATABASE_MODEL.md`	One bullet added documenting `onboarding_completed` and the new migration.

D. Exact Database Migration Created and What It Does

`supabase/migrations/20260910000000_add_onboarding_completed.sql` — timestamp is after the existing `20260909000000_create_profiles.sql`, so migration history is additive and nothing is edited destructively.

1. Schema:

```
alter table public.profiles
    add column if not exists onboarding_completed boolean not null default false;
```

2. Backfill (deliberately *not* "mark every existing profile completed"):

```
update public.profiles
set onboarding_completed = true
where onboarding_completed = false
  and age_range is not null
  and gender is not null
  and is_skin_sensitive is not null;
```

Choice and rationale: In the shipped app the only writer of `age_range`, `gender`, and `is_skin_sensitive` is the final onboarding save, which writes them together. Their combined presence is therefore reliable evidence that the questionnaire was completed. `skin_type` is **excluded** from the condition because the onboarding flow lets the user tap "Skip for now", leaving it legitimately `null`. Any row missing a required answer keeps the safe default `false`, so that user is asked the onboarding questions again rather than being silently advanced past onboarding with incomplete data. A false-negative here is harmless (ask once more); a false-positive would strand a user with `null` answers.

3. RLS: unchanged. Adding a column does not alter existing policies; the per-user `SELECT` / `INSERT` / `UPDATE` policies from the initial migration continue to restrict every row to `auth.uid() = id`. No new policy is required because `onboarding_completed` lives on the row the user already owns. The one-time `UPDATE` runs with migration privileges server-side — never from the client, and never with a service-role key embedded in the Android app.

4. Forward / backward safety: If the app runs before this migration is applied, `ProfileRow.onboarding_completed` deserializes to its default `false` (Kotlinx serialization default + Supabase's `ignoreUnknownKeys`), so nothing crashes and every user is simply treated as "not yet completed" — the safe direction.

E. How Each Onboarding Answer Is Persisted

Answers are collected into the single app-scoped `OnboardingViewModel.uiState` (`OnboardingUiState`) as the user moves through the existing screens:

Screen (unchanged UI)	Callback	`OnboardingUiState` field	Column
`AgeSelectionScreen`	`setAge(String)`	`ageRange`	`age_range`
`SkinTypeScreen` (incl. "Skip for now" → `null`)	`setSkinType(String?)`	`skinType`	`skin_type`
`GenderSelectionScreen`	`setGender(String)`	`gender`	`gender`
`SkinSensitivityScreen` (final question)	`setSensitivityAndSave(Boolean)`	`isSkinSensitive`, then triggers `save()`	`is_skin_sensitive`
(from signup / provider metadata)	`start(name)`	`fullName`	`full_name`

`save()` builds **one** `UserProfile(id = currentUserId(), fullName, ageRange, skinType, gender, isSkinSensitive, onboardingCompleted = true)` and calls `profileRepository.saveProfile(...)`.

`SupabaseProfileRepository.saveProfile` performs a single `client.postgrest["profiles"].upsert(profile.toProfileRow())` — an upsert on primary key `id`, writing `full_name, age_range, skin_type, gender, is_skin_sensitive, onboarding_completed` (`created_at` / `updated_at` are DB-managed).

On `onFailure`: `errorMessage` is set, `isSaving = false`, `saveCompleted` stays `false`, no navigation. On `onSuccess`: `saveCompleted = true`, which drives navigation to Welcome → Home.

Data-loss items from the audit are closed: `full_name` is now populated on every entry path via `start()`, and completion is an explicit column rather than an inferred one.

F. How the Authenticated Supabase User ID Is Associated with the Profile

`OnboardingViewModel.save()` calls `authRepository.currentUserId()` → `SupabaseAuthRepository.currentUserId()` → `client.auth.currentUserOrNull()?.id`.

- If it is `null`, `save()` sets `errorMessage = "Your session is no longer active."` and **returns without calling the repository** — no persistence.
- Otherwise that id is `UserProfile.id` → `ProfileRow.id` → the upsert's primary key.

The UI layer never supplies, and cannot override, an id — screens only pass answer values through typed callbacks. Server-side, RLS `with check ((select auth.uid()) = id)` rejects any insert / update whose row id is not the caller's own. `ProfileViewModel.refresh()` likewise reads `authRepository.currentUserId()` itself and never accepts an id from a screen.

G. How Onboarding Completion Is Now Determined

`SupabaseProfileRepository.hasCompletedOnboarding(userId)`:

```
client.postgrest["profiles"]
    .select { filter { eq("id", userId) } }
    .decodeList<ProfileRow>()
    .isOnboardingCompleted()          // firstOrNull()?.onboarding_completed == true
```

A row that exists with `onboarding_completed = false` returns **`false`**. No row returns `false`. `AuthViewModel.determineDestination()` (unchanged) consumes this `Result<Boolean>` to choose `HOME` vs `ONBOARDING`, and it is evaluated both at startup (`startupDestination`) and on a fresh `AUTHENTICATED` transition (`AuthEvent.SignedIn(destination)`). The decision rule is a pure `internal` function so it is unit-tested directly without a Supabase client.

H. How Manual-Signup and Google Display Names Are Handled

- **Manual email signup:** `SignUpScreen` → `AuthViewModel.signUp(fullName, …)` → `AuthEvent.SignedUp(fullName)` → `WeGlowApp` calls `onboardingViewModel.start(event.fullName)`. The trimmed, validated, non-blank name goes into `OnboardingUiState.fullName` and is persisted to `profiles.full_name` by the final save. Supabase email signup writes no user metadata, so the metadata fallback cannot override it. This path is unchanged except that `start` is now null-safe.
- **Email sign-in of an incomplete user / Google new-or-incomplete user / restored session into onboarding:** `WeGlowApp` now calls `onboardingViewModel.start()` (no argument) at each of these entries. `start()` first wipes any prior `OnboardingUiState`, then resolves the name from `authRepository.currentUserDisplayName()` → `SupabaseAuthRepository` reads `userMetadata` keys `full_name` → `name` → `user_name` → `preferred_username`, first non-blank wins (Google populates `full_name` / `name`). If nothing usable exists, `fullName` is `""` and persists as `null`.
- **No invented names.** The name is only ever a value the user typed at signup or a value the identity provider returned.
- The resolved name is persisted to `profiles.full_name` as part of the same successful onboarding upsert.

I. How Home / Profile Obtain the Real User's Name

New `ProfileViewModel` (constructed in `WeGlowApp` via the existing `viewModelFactory` helper from `container.authRepository` + `container.profileRepository`; `AppContainer` needed no change — it already exposes both).

`WeGlowApp` runs `LaunchedEffect(Unit) { profileViewModel.refresh() }` when the Home and Profile destinations are entered. `refresh()`:

1. `authRepository.currentUserId()` — if `null`, clears state so the neutral fallback shows.
2. `profileRepository.getProfile(userId)` → `SupabaseProfileRepository` → `select … decodeList<ProfileRow>().firstOrNull()?.toUserProfile()`.
3. `displayName = persistedProfile.fullName?.trim()?.takeIf { it.isNotBlank() } ?: authRepository.currentUserDisplayName()?.trim()?.takeIf { it.isNotBlank() }`.

`WeGlowApp` passes `profileState.displayName` into `HomeScreen(displayName = …)` and `ProfileScreen(displayName = …)`.

- Home: `displayName?.let { "Hello, $it" } ?: "Hello there"`.
- Profile: `displayName ?: "Your Profile"`.

Both are neutral fallbacks — no fabricated user data. No Supabase import enters `feature/` or `ui/`. Figma layout, spacing, styling, navigation, and other functionality of both screens are untouched. No profile-photo upload (Phase 6).

J. Tests Added / Updated and What They Verify

`OnboardingViewModelTest.kt` (rewritten; fakes now satisfy the full Phase 4 `AuthRepository` + new members)

Test	Verifies (maps to the requested list)
`finalAnswer_savesAllAnswersForSignedInUser`	All four answers + **`id = currentUserId()`** + **`onboardingCompleted = true`** in the saved `UserProfile`; `saveCompleted = true`; `isSaving = false`. → (1)(2)(3)
`successfulSave_marksOnboardingCompletedAndSaveCompleted`	`onboardingCompleted = true` persisted; a skipped skin type stays `null`; `saveCompleted = true`; no error. → (2)(3)

`failedSave_doesNotProduceSaveCompleted`	Repository returns `Result.failure` → `saveCompleted = false`, `isSaving = false`, `errorMessage` retained; the attempted payload still carried `onboardingCompleted = true` (completion is only "real" once persisted). → (4)
`missingAuthenticatedUser_doesNotSave`	`currentUserId() == null` → repository never invoked, `saveCompleted = false`, error set. → (5)
`start_withoutManualName_usesAuthenticatedDisplayName`	`start()` with no arg pulls `"Google Person"` from provider metadata into the persisted `full_name`. → (H)
`start_withoutNameOrMetadata_persistsNullName`	No name anywhere → `full_name` persists as `null`, never invented. → (H)
`start_clearsStaleAnswersFromPreviousAttempt`	A second `start("Second User")` wipes prior age / skin / gender / sensitivity and `saveCompleted`. → identity-leak safety

`SupabaseProfileRepositoryTest.kt` (new; pure — no Supabase client needed)

Test	Verifies
`noRow_isNotCompleted`	Empty result → `false`.
`rowExistsButFlagFalse_isNotCompleted`	A fully-populated row with `onboarding_completed = false` → **`false`**. Directly proves completion is decided by the flag, not by row existence. → (6)
`rowWithFlagTrue_isCompleted`	Flag `true` → `true`.
`profileRow_mapsToUserProfile`	Field-by-field `ProfileRow` → `UserProfile` mapping including the new flag. → (7)
`userProfile_roundTripsThroughProfileRow`	`toProfileRow().toUserProfile()` is identity. → (7)
`defaultProfileRow_isNotCompleted`	Defaults are safe (`skin_type` null, flag false).

`AuthViewModelTest.kt`

- Compile fixes for the new interface members on both fakes; `FakeProfileRepository` can hold a `storedProfile` and derives completion from it.
- **Added** `restoredSession_withProfileRowButIncompleteOnboarding_routesToOnboarding` — a stored profile row with `onboardingCompleted = false` routes a restored session to `ONBOARDING` (behavioral coverage of (6) through `determineDestination`).
- All 8 pre-existing tests retained and pass (9 total).

`ArchitectureFoundationTest.kt`

- All original assertions kept; one assertion added for `onboarding_completed` mapping.

No valid existing test was removed to make the build green.

K. Exact Result of `./gradlew clean architectureCheck testDebugUnitTest lintDebug assembleDebug`

Run with `export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"` (system `java` is not on PATH on this Mac).

Per-task outcome (captured with `--continue` so every task executed):

```
> Task :app:architectureCheck            SUCCESS
> Task :app:compileDebugKotlin           SUCCESS   (pre-existing deprecation warnings only, in ScanScreen.kt /
ScanResultScreen.kt — not touched)
> Task :app:compileDebugUnitTestKotlin   SUCCESS   (previously did not compile; audit blocker B4 fixed)
> Task :app:assembleDebug                SUCCESS   -> app/build/outputs/apk/debug/app-debug.apk written
> Task :app:lintDebug                    SUCCESS   -> lint-results-debug: 0 errors, 87 warnings (all pre-existing; 0 in
any Phase 5 file)
> Task :app:testDebugUnitTest            FAILED    -> 34 tests completed, 1 failed

BUILD FAILED     (solely because of the single failing test below)
```

Per-suite unit-test results:

Suite	Result
`feature.onboarding.OnboardingViewModelTest`	**7 / 7 pass**

`data.repository.SupabaseProfileRepositoryTest`	**6 / 6 pass**
`feature.auth.AuthViewModelTest`	**9 / 9 pass**
`ArchitectureFoundationTest`	**1 / 1 pass**
`feature.scan.ScanViewModelTest`	**3 / 3 pass**
`data.scan.BundledAcneModelTest`	**1 / 1 pass**
`ExampleUnitTest`	**1 / 1 pass**
`data.scan.YoloPostProcessorTest`	**5 / 6 pass — 1 failure**

Failing test: `com.example.weglow.data.scan.YoloPostProcessorTest > ignoresBelowThresholdInvalidAndPaddingOnlyDetections` — `java.lang.AssertionError` at `YoloPostProcessorTest.kt:37` (`assertTrue(detections.isEmpty())`).

A full PASS is not claimed. Every Phase 5 / onboarding / auth / profile / mapping check passes; `architectureCheck`, `lintDebug`, and `assembleDebug` pass; the overall command exits non-zero because of the one scan test described in section M.

L. What You Must Do Manually in the Supabase Dashboard

1. **Apply the new migration** `supabase/migrations/20260910000000_add_onboarding_completed.sql` to your project — either `supabase db push`, or open the SQL Editor and run the file's contents. It adds the `onboarding_completed` column, runs the conservative backfill, and leaves RLS unchanged.
2. Nothing else is required: no policy edits, no table changes, no dashboard configuration, no keys. Do **not** add a service-role key to the Android app.
3. Optional, for Google display names to populate `full_name`: confirm your Google OAuth provider returns the basic profile scope (Supabase's default Google setup already does), so `user_metadata.full_name` / `name` is present on the session.

M. Remaining True Blocker

`YoloPostProcessorTest.ignoresBelowThresholdInvalidAndPaddingOnlyDetections` fails — pre-existing, in teammate scan code, not caused by Phase 5, and outside the scope the instructions forbid modifying.

- `app/src/main/java/com/example/weglow/data/scan/YoloPostProcessor.kt` was last changed in commit `7b4b8bb` ("Show low-confidence detections without confidence labels"), which deliberately changed low-confidence detection handling. `app/src/test/java/com/example/weglow/data/scan/YoloPostProcessorTest.kt` was last changed in the earlier commit `945ec34`; its expectation (`detections.isEmpty()` for an anchor whose class score is exactly `0.25`) was never updated to match the new processor behavior.
- The failure was **masked** until now: on a clean checkout the unit-test source set does not compile, because `OnboardingViewModelTest`'s fake `AuthRepository` was missing `authenticationState` and `signInWithGoogle` (audit blocker B4). With no compilation, `testDebugUnitTest` never executed a single test. Phase 5 repaired that compile blocker, which is why this stale scan test is now visible and running.
- Per the constraints (do not modify the acne detection / model / scan implementation unless a *compile* error is caused by Phase 5; do not refactor teammate scan systems; preserve all scan work; do not hide failing tests), both `YoloPostProcessor.kt` and `YoloPostProcessorTest.kt` were left untouched and the failure is reported here.
- The fix is one line and belongs to the scan feature owner — either update the test's `assertTrue(detections.isEmpty())` expectation to match the intended low-confidence behavior, or re-introduce the sub-threshold filter in `YoloPostProcessor.decode`. Until then, `./gradlew … testDebugUnitTest …` will exit non-zero on this single test while all Phase 5 verification passes.

No other blockers. No accidental or unrelated modifications are present in the working tree — `git status` shows only the files listed in sections B and C.